JTPC: Joint Timeliness and Power Control for Real-Time Embedded GPU Systems

***Abstract*—GPUs have been widely adopted in *real-time* embedded systems to support compute-intensive applications such as autonomous driving and virtual reality rendering. Traditional GPU scheduling solutions rely heavily on offline schedulability analysis and therefore cannot adapt to runtime workload variations. Recent adaptive approaches, such as Response Time Ratio (RTR) control, improve deadline compliance by dynamically adjusting task rates in response to changing workload conditions. However, these approaches focus exclusively on timing guarantees and do not consider GPU power consumption, a critical constraint for embedded platforms operating under strict power and thermal budgets. Regulating power independently is also nontrivial, since task rate adaptation and GPU frequency scaling are inherently coupled: changing task rates affects GPU power consumption, while frequency scaling directly impacts task execution times.**

To address the coupling between timing and power consumption, we derive a Multi-Input Multi-Output (MIMO) system model that captures GPU resource contention among co-scheduled tasks and power consumption. Based on this model, we design `JTPC`, an integrated feedback-control framework that jointly controls the task response time and GPU power consumption via rate adaptation and dynamic frequency scaling. We implement `JTPC` on an embedded GPU platform and evaluate it under runtime workload variations. Experimental results show that `JTPC` maintains timing guarantees while keeping GPU power within the specified budget, significantly reducing deadline misses and power violations compared to state-of-the-art baselines.

***Index Terms*—GPU scheduling, response time ratio control, rate adaptation, and feedback control.**

I. Introduction

Over the past decade, Graphics Processing Units (GPUs) have fundamentally transformed the fields of Artificial Intelligence (AI) and Machine Learning (ML). Due to their ability to accelerate parallel computation, they are widely used in modern real-time embedded applications, such as virtual reality video rendering [1], object detection [2], and autonomous vehicle path planning [3]. To facilitate these real-world applications, scheduling real-time tasks on GPUs has recently become an important area of research and has received considerable attention [4]–[10].

Traditional approaches to scheduling *real-time* tasks rely on *open-loop* schedulability analysis, which requires prior knowledge of the Worst-Case Execution Times (WCETs) of the tasks to ensure that deadlines are met [11]–[14]. While these approaches may be effective in systems with predictable workloads, they may lead to deadline violations or significant resource underutilization when task execution times vary dynamically. For example, an autonomous vehicle [3] may need to activate an additional perception task for obstacle detection in response to unexpected environmental events, or a drone may encounter increased scene complexity requiring more intensive vision processing. Such runtime changes introduce additional GPU workloads or increase the computational demand of existing tasks, leading to greater contention for shared GPU resources, including compute cores, memory bandwidth, and scheduling queues. As a result, GPU execution times can vary significantly, causing statically configured approaches to either violate timing constraints [9] or severely underutilize system [5], [7], [15], [16] resources. Consequently, *real-time* GPU applications require adaptive runtime mechanisms that can dynamically respond to changing execution conditions while preserving timing guarantees.

Recently, feedback control techniques have shown strong potential for providing *real-time* guarantees by dynamically adapting system behavior to workload variations. In particular, controlling GPU task Response Time Ratio (`RTR`), which is defined as the measured task response time divided by its relative deadline [9], has demonstrated effectiveness for soft *real-time* systems. The primary objective of `RTR` control is to keep task response times within their deadline constraints despite runtime workload changes. However, existing `RTR` control approaches focus solely on timeliness guarantees without explicit GPU power management. Unfortunately, the higher densities of memory and computing units of GPUs can result in a higher chance of violating their Thermal Design Power (TDP) limits and thus having thermal failures [17], [18]. Overheating can cause a sudden drop in GPU's performance, lag, or even a system crash or shutdown, particularly for embedded GPUs that are often deployed in compact spaces without fans [19], [20]. Therefore, GPU power consumption must also be explicitly controlled for improving system reliability and reducing expensive cooling requirements [20].

One common technique for keeping GPU power consumption within the platform's prescribed TDP limit is through runtime power control, which is widely used to ensure that the power consumption of data center GPUs stays below their desirable power budgets for preventing overheating [21], [22]. Similar mechanisms must also be adopted in embedded GPU platforms to meet strict power and thermal constraints. In practice, power control is often implemented via Dynamic Frequency Scaling (DFS), in which the GPU operating frequency can be reduced to lower power consumption [22]–[24]. However, throttling the GPU frequency can have significant negative impacts on application performance [23]. This is particularly problematic in *real-time* systems, where tasks have stringent deadline requirements and even minor increases in response time can lead to deadline misses.

To have explicit control of both task timeliness and GPU power consumption, one possible approach is to regulate them separately with two different control loops. For example, prior work has proposed feedback-based response-time control by adapting the invocation rates of periodic tasks within allowed ranges (i.e., rate adaptation) [9], while other studies have used DFS for GPU power control [21]. However, simultaneously controlling response time and power consumption is challenging because these control objectives are tightly coupled (Section III). Specifically, changing GPU frequency to regulate power consumption directly affects task execution times, while rate adaptation also affects power consumption. As a result, independently designed control loops can interfere with each other, degrading control performance, causing deadline misses, or violating power limits. The second approach employs a multi-tier control strategy with coordination, in which the two control loops operate at significantly different time scales. This separation reduces their interactions, allowing the residual coupling effects to be modeled as noise [25]. Unfortunately, neither power nor response time can be monitored and controlled on a long time scale, because violating either constraint can cause violations. Therefore, an integrated control framework is required to jointly control task response times and GPU power consumption, explicitly, accounting for the cross-coupled interactions between these control objectives.

In this paper, we present JTPC, a joint timeliness and power control framework for soft *real-time* embedded GPU systems. Unlike prior feedback-control approaches that regulate either power consumption or response-time ratio independently, JTPC simultaneously ensures that each task meets its timing constraints while enforcing a system-level power set point. A key novelty of JTPC is a Multi-Input Multi-Output (MIMO) system model that is analytically developed to characterize the complex impacts of rate adaptation and DFS on both task scheduling and GPU power consumption. Based on this MIMO model, JTPC is systematically designed using feedback control theory, enabling joint regulation of timeliness and power while providing formal guarantees on system stability and control performance.

Specifically, this paper makes the following contributions:

- We identify and formalize the problem of jointly controlling task response time and GPU power consumption in *real-time* embedded GPU systems, highlighting the coupling between task-rate adaptation and DFS.
- We design JTPC based on MIMO control theory for guaranteed system stability and control accuracy. The design choices of each control component are discussed in detail.
- We perform extensive experiments on a hardware testbed with an Nvidia Jetson embedded GPU to show that JTPC not only achieves the desired control performance, but also outperforms several state-of-the-art solutions by having fewer deadline misses and less power violation.

The rest of the paper is organized as follows. Section II discusses the task model and the problem formulation. Section III proposes the System Architecture. Section IV provides the controller design. Section V describes the implementation and experimental setup. The hardware experiments are performed in Section VI, and Section VII discusses the related work. Section VIII concludes the paper.

II. Problem Formulation and Motivation

In this subsection, we define our task model and formulate our control problem.

A. Task Model

In *real-time* systems, a task is implemented as a process or thread that must be scheduled to execute on the target GPU at particular time intervals. Each *real-time* task in our system is realized as a process or thread whose work must execute on the GPU at specific points in time. A task releases a sequence of instances, called jobs, and the release time of a job marks the moment it becomes available for scheduling on the hardware. To keep the design simple, all tasks are assumed to be periodic. This choice reflects how embedded *real-time* software is typically structured: components are activated at regular intervals to interact with sensors, cameras, and other parts of the environment [26]. We further adopt the implicit-deadline assumption, under which the deadline of every job equals the period of its task. The notation used in the rest of the paper is listed below. The system runs N tasks τ_i ($1 \leq i \leq N$) with a controller that performs actuator updates once every control period (where the control period is the elapsed time between the control decisions). Each task τ_i is characterized by the following parameters:

- $r_i(k)$: Task rate of τ_i in the kth control period.
- θ_i: Context switching overhead of task τ_i.
- $r_{\min}^{(i)}$, $r_{\max}^{(i)}$: Minimum and maximum rates for task τ_i.
- e_i: Estimated execution time of τ_i on the GPU.
- $p_i(k) = 1/r_i(k)$: Period of τ_i in the kth control period.
- $d_i(k)$: Deadline of τ_i (assumed $d_i(k) = p_i(k)$).
- $f(k)$: GPU frequency in the kth control period.
- $f_{\min}$, $f_{\max}$: Minimum and maximum GPU frequencies.
- $q_i(k)$: Response time of τ_i in the kth control period.
- z_i: RTR set point for τ_i.
- P_s: Power set point for the GPU.
- $P_g(k)$: GPU power in the kth period.
- P_{gs}: Static power consumption of the GPU.

This task model is applicable to a wide range of periodic tasks. We represent each periodic task as $\tau_i(e_i, p_i)$, where the execution time e_i and the period p_i are expressed in milliseconds. The RTR of a task is shown in Equation (1).

$$rtr_i(k) = \frac{q_i(k)}{p_i(k)} = q_i(k) \cdot r_i(k). \tag{1}$$

An RTR of 1 indicates that a job finished exactly on its deadline, while $rtr_i(k) > 1$ signifies a deadline miss. For example, with an RTR set point of 0.6 and a period of 2 s, each job should finish by 1.2 s. Our task model has several properties. (a) First, while schedulability analysis provides an estimated execution time for each task, its actual execution

time may vary at runtime due to GPU frequency scaling [22], [24] and undisclosed GPU scheduling [6], [8]. Although prior work has attempted to reverse engineer GPU schedulers [9], [11], [14], [27], the exact runtime allocation and scheduling behavior of co-scheduled tasks remains largely unknown. As a result, task execution times can vary significantly at runtime. (b) The rate of a task may be dynamically adjusted within a range [9], [28]–[32]. This assumption is based on the fact that the task rates in many applications (e.g., camera sensors [33] and multimedia [34]) can be dynamically adjusted without causing system failure. In addition, we assume the task rates are normalized. (c) The GPU frequency can be adjusted within certain bounds [21], [22], [24] and most GPUs provide interfaces through drivers to update the GPU operating frequency [17], [35]. Additionally, we assume the frequency ranges are continuous because a GPU can approximate any continuous value with a series of discrete frequency levels (see Section V). (d) We assume each task is launched from a separate process, and when the tasks are launched from separate processes, the GPU time-slices them [11], [12], [27].

B. Problem Formulation

The optimization formulation minimizes the differences between the RTR of each task and its corresponding set point, and between the GPU power consumption and its corresponding set point, by adjusting the task rate of each task and the GPU frequency subject to their respective constraints. The optimization problem is defined as:

$$\begin{aligned} \min_{\{r_i(k)\},f(k)} \quad & \sum_{i=1}^{N}\left(\frac{z_i - rtr_i(k)}{z_i}\right)^2 + \left(\frac{P_s - P_g(k)}{P_s}\right)^2 \\ \text{s.t.} \quad & r_{\min}^{(i)} \le r_i(k) \le r_{\max}^{(i)}, \quad \forall i = 1,\ldots,N, \\ & f_{\min} \le f(k) \le f_{\max}. \end{aligned} \tag{2}$$

The constraints ensure that task rates $r_i(k)$ remain within their allowable bounds $[r_{\min}(i), r_{\max}(i)]$, while GPU frequency scaling is restricted to the range $[f_{\min}, f_{\max}]$. These bounds are enforced at runtime by clamping the controller outputs to the corresponding minimum and maximum values before applying them to the system. Formally, the scheduling objective is an *offline* optimization problem. Rather than using traditional *offline* methods, we adopt a control-theoretic strategy that solves it *online* using a *closed-loop* feedback system. The *closed-loop* solution suits *real-time* GPU scheduling because it responds immediately to changes and remains robust to workload fluctuations and varying execution times [9], [25].

III. System Overview

A. System Modeling

To apply a control-theoretic approach, we first develop a dynamic model that describes the relationship between the manipulated variables ($r_i(k)$ and $f(k)$) and the controlled variables (rtr_i and P_g). Controlled variables represent the system outputs that the controller regulates to desired set points, such as RTR and GPU power consumption. Manipulated variables represent the system parameters that the controller dynamically adjusts to influence the controlled variables.

As observed in previous research [9], when tasks are launched from separate processes, the response time of one task is significantly affected by the rate/frequency of the co-scheduled task. For a system with N tasks, the response time is computed as shown below

$$q_i(k) = e_i + \sum_{\substack{j=1 \\ j\neq i}}^{N}(e_j + \theta_j)\cdot r_j(k-1), \qquad \forall i = 1,\ldots,N. \tag{3}$$

The GPU power consumption comprises both static (idle) and dynamic components [24], [36], [37]. The static power consumption is the baseline power drawn by the GPU when idle, including components such as the power supply and fans. The dynamic power consumption varies with workload and is directly proportional to the GPU's operating frequency. Thus, the total power consumption of a GPU is represented in Equation (4), where $P_g(k)$ is the total power consumption in the kth control period. P_{gs} represents the static power consumption (obtained from measurements of the GPU's power draw at idle), α is the coefficient for linear dynamic power contribution, and f is the GPU's operational frequency in Megahertz.

$$P_{\mathrm{g}}(k) \;=\; P_{gs} \;+\; \alpha \;\cdot f(k-1). \tag{4}$$

Prior research shows that GPU task execution time scales inversely with GPU frequency [21], [38], [39], meaning that lowering the GPU frequency increases task response times. Furthermore, the task rates of co-scheduled tasks also influence GPU power consumption. Specifically, smaller task periods (i.e., higher task rates) lead to more frequent task releases, which increase GPU utilization and resource contention and, in turn, GPU power consumption. To verify and quantify these cross-coupled effects, we conduct controlled experiments on the target platform. Figure 1(a) shows that increasing GPU frequency reduces task response time by accelerating GPU execution.

Since task execution time scales inversely with GPU frequency, reducing the GPU frequency slows task execution and increases response time, whereas increasing the GPU frequency accelerates execution and reduces response time. The magnitude of this effect is quantified by a frequency-sensitivity coefficient that characterizes the extent to which the task response time varies with GPU frequency. The updated response-time model is given by

$$q_i(k) = e_i + \sum_{\substack{j=1 \\ j\neq i}}^{N}(e_j+\theta_j)\cdot r_j(k-1) - \beta_i f(k-1), \forall i = 1,\ldots,N. \tag{5}$$

where β_i denotes the slowdown sensitivity of task i with respect to GPU frequency.

Likewise, to capture the effect of task rates on GPU power consumption, we extend the power model with a workload-dependent term. Figure 1(b) shows that shorter task periods

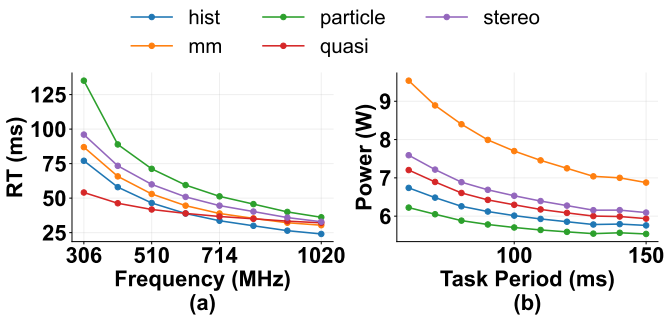


Fig. 1: Impact of control inputs on system behavior: (a) task response time as a function of GPU frequency, showing that higher GPU frequencies reduce response time and (b) GPU power consumption as a function of task period, showing that shorter periods (higher task rates) increase power consumption

(i.e., higher task rates) lead to more frequent task releases, which increase GPU utilization and resource contention and, in turn, dynamic power consumption. The extent to which each task contributes to GPU power consumption is characterized by a task-rate sensitivity coefficient. The updated GPU power model is expressed as

$$P_g(k) = P_{gs} + \alpha f(k-1) + \sum_{i=1}^{N} c_i \cdot r_i(k-1), \quad (6)$$

where c_i denotes the sensitivity of GPU power consumption to the rate of task t_i. A higher task rate increases GPU activity and, therefore, increases GPU power consumption.

To facilitate controller design later, we derive a difference equation from the model in Equations (5) and (6), using the steps below.

$$q_i(k-1) = e_i + \sum_{\substack{j=1 \\ j\neq i}}^{N} (e_j + \theta_j) \cdot r_j(k-2) - \beta_i \cdot f(k-2), \qquad \forall i = 1, \ldots, N. \quad (7)$$

$$P_g(k-1) = P_{gs} + \alpha f(k-2) + \sum_{i=1}^{N} c_i \cdot r_i(k-2), \quad (8)$$

Subtracting Equation (5) from Equation (7), and subtracting Equation (6) from Equation (8) we obtain

$$q_i(k) - q_i(k-1) = \sum_{\substack{j=1 \\ j\neq i}}^{N} (e_j + \theta_j)\big(r_j(k-1) - r_j(k-2)\big) - \beta_i\big(f(k-1) - f(k-2)\big), \forall i = 1, \ldots, N. \quad (9)$$

$$P_g(k) - P_g(k-1) = \alpha\big(f(k-1) - f(k-2)\big) + \sum_{i=1}^{N} c_i\big(r_i(k-1) - r_i(k-2)\big). \quad (10)$$

Let

$$\Delta r_i(k-1) = r_i(k-1) - r_i(k-2) \quad (11)$$

$$\Delta f(k-1) = f(k-1) - f(k-2) \quad (12)$$

Using these definitions, Equations (9) and (10) can be rewritten as

$$q_i(k) = q_i(k-1) + \sum_{\substack{j=1 \\ j\neq i}}^{N} (e_j + \theta_j)\Delta r_j(k-1) - \beta_i \Delta f(k-1), \qquad \forall i = 1, \ldots, N. \quad (13)$$

$$P_g(k) = P_g(k-1) + \sum_{i=1}^{N} c_i \Delta r_i(k-1) + \alpha \Delta f(k-1). \quad (14)$$

The combined response-time and power model can then be written in the following MIMO form:

$$\mathbf{y}(k) = \mathbf{A} \cdot \mathbf{y}(k-1) + \mathbf{B} \cdot \mathbf{u}(k-1), \quad (15)$$

where

$$\mathbf{y}(k) = \begin{bmatrix} q_1(k) \\ q_2(k) \\ \vdots \\ q_N(k) \\ P_g(k) \end{bmatrix}, \qquad \mathbf{u}(k-1) = \begin{bmatrix} \Delta r_1(k-1) \\ \Delta r_2(k-1) \\ \vdots \\ \Delta r_N(k-1) \\ \Delta f(k-1) \end{bmatrix}. \quad (16)$$

and $\mathbf{A} = \mathbf{I}$. The system matrix $\mathbf{B}$ is given by

$$\mathbf{B} = \begin{bmatrix} 0 & \tilde{e_2} & \cdots & \tilde{e_N} & -\beta_1 \\ \tilde{e_1} & 0 & \cdots & \tilde{e_N} & -\beta_2 \\ \vdots & \vdots & \ddots & \vdots & \vdots \\ \tilde{e_1} & \tilde{e_2} & \cdots & 0 & -\beta_N \\ c_1 & c_2 & \cdots & c_N & \alpha \end{bmatrix}, \quad (17)$$

where $\tilde{e_i} = e_i + \theta_i$.

The coefficients in $\mathbf{B}$ can be obtained from a combination of analytical modeling and system identification. The impact of task rate on response-time is derived from the response-time model in Equation (5), which is used in prior work such as FC-GPU [9]. However, no established analytical model exists for the effects of GPU frequency on task response time or for the effects of task rates and GPU frequency on GPU power consumption in our target system. Therefore, we use a standard approach to this problem called system identification [22], [40], [41]. Specifically, we estimate the system parameters from measurements collected during the experiments and use them to construct an empirical model of the input-output behavior. The frequency-sensitivity coefficients $\beta_1, \ldots, \beta_N$ are obtained by sweeping GPU frequency and measuring the corresponding response-time changes for each task, and the power-sensitivity coefficients $c_1, \ldots, c_N$ are obtained by varying task periods (and hence task rates) while measuring GPU power consumption. Finally, the frequency-to-power coefficient α is obtained by measuring GPU power consumption across different GPU frequencies. These coefficients populate the input matrix $\mathbf{B}$ in Equation (17).

The identified model in Equation (15), obtained from system identification, is validated using a step-like input signal.

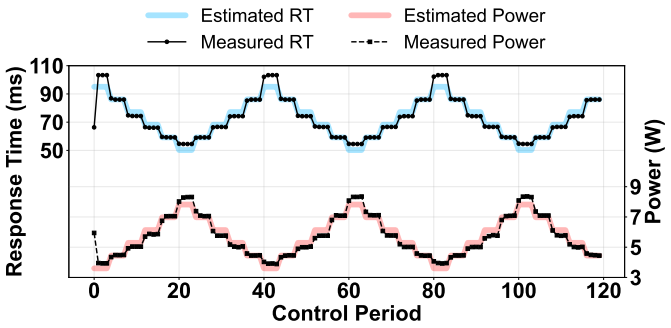


Fig. 2: Validation of the system model showing the measured and predicted response time and power consumption.

Figure 2 shows that the predicted values are closer to the measured values. Therefore, we use the model in Equation (15) for our controller design. Although the identified model may not perfectly capture all system behaviors, the use of a feedback controller enables the system to compensate for modeling inaccuracies and runtime disturbances. As a result, the controller can still achieve the desired control objectives even in the presence of model uncertainty.

B. System Architecture

Although either control knobs can be used to independently control the system states, both are used in a joint approach to allow for independent and precise control as well as a larger operating range. Several approaches can be used to control RTR and GPU power consumption. A straightforward approach uses two separate control loops: one loop adjusts task rates to regulate RTR, while the other loop adjusts GPU frequency to regulate GPU power consumption. However, such a separated design introduces undesired mutual interference because task rates, GPU frequency, RTR, and GPU power consumption are coupled, as shown in Equation (17). Specifically, changing the GPU frequency affects not only GPU power consumption but also task execution times and, consequently, RTR. Similarly, changing task rates affects not only RTR but also GPU utilization and, therefore, GPU power consumption. As a result, the two control objectives are inherently coupled, requiring a unified joint control architecture. In this subsection, we present the architecture of JTPC. The control objective of JTPC is to jointly regulate the RTR of GPU tasks and GPU power consumption while preserving real-time performance and satisfying the platform power set point.

The joint control architecture of JTPC is shown in Figure 3. To perform rate adaptation and dynamic frequency scaling, the JTPC architecture features a MIMO feedback control loop that dynamically adjusts task rates to ensure that the task RTRs converge to their specified set points, and the GPU power consumption can also be controlled to its set point. The MIMO controller is located on the host CPU. A periodic feedback loop is invoked at every sampling instant and comprises a *Response time monitor*, *Power monitor* , *controller*, *Task rate actuator* and *frequency actuator*. The process within each control period is as follows:

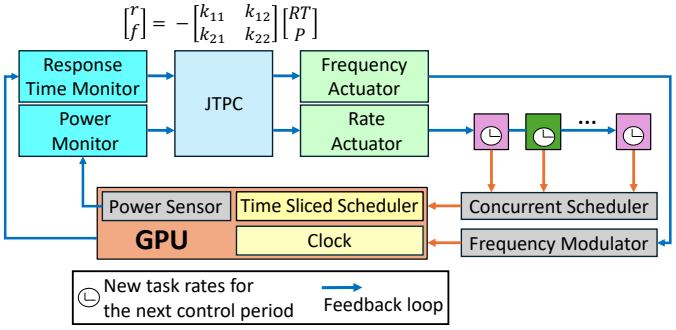


Fig. 3: Overall framework of the JTPC control loop. JTPC periodically monitors the RTR of each task and GPU power consumption, and jointly adjusts task rates and GPU frequency to keep task response times below their deadlines while maintaining GPU power consumption within the specified power set point.

1) The monitor measures the response time ratio for each task ($rtr_i(k)$) and the power consumption of the GPU ($P_g(k)$) during the last control period.
2) The controller collects the RTR values $rtr_i(k)$, the input task rates $r_i(k)$, the GPU power consumption $P_g(k)$, and the GPU frequency $f(k)$. Using these values, the controller computes the required control adjustments to the task rates and the GPU frequency.
3) The Task Rate Actuator applies the updated task rates $r_i(k+1)$, while the GPU Frequency Modulator applies the updated GPU frequency $f(k+1)$ to regulate the system behavior during the next control period.

IV. Controller Design and Analysis

A. State space representation

Based on the system model, a MIMO controller can be designed to ensure that the RTR settles at its desired set point and that the GPU power consumption is capped at its set point. The PID control approach used in earlier works on real-time feedback control scheduling [9], [42] is suitable for real-time systems. The State space representation of a MIMO controller is shown in Equation (18).

$$\begin{aligned} x(k+1) &= Ax(k) + Bu(k) \\ y(k) &= Cx(k) + Du(k) \end{aligned} \tag{18}$$

Here- $x(k)$ is the state vector, $u(k)$ is the input vector, $y(k)$ is the output vector, A, B, C, and D are matrices that define the system dynamics and output. In our system, we take B to be the matrix from the system model, C to be the identity matrix, and D to be 0.

In our control problem, the controlled variables are the response time ratios $rtr_i(k)$, while the manipulated variables are the task rates $r_i(k)$ and GPU frequency $f(k)$. However, the model in Equation (15) computes the response times of tasks rather than the RTR values directly. Since the RTR of a task is defined as the ratio of its response time to the task

period, we modify the output matrix C to map the response-time outputs to RTR values. Therefore, we define the output matrix C as

$$C = \begin{bmatrix} \frac{1}{p_1(k)} & 0 & \cdots & 0 \\ 0 & \frac{1}{p_2(k)} & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \frac{1}{p_N(k)} \end{bmatrix}, \tag{19}$$

where each diagonal element converts the corresponding task response time into its RTR value. We use this modified output matrix in the controller design so that the controller directly regulates RTR rather than response time. The final representation of our MIMO system is shown in Equation (20).

$$\begin{bmatrix} x(k+1) \\ y(k) \end{bmatrix} = \begin{bmatrix} I & B \\ C & 0 \end{bmatrix} \begin{bmatrix} x(k) \\ u(k) \end{bmatrix} \tag{20}$$

B. LQR Control

We apply Linear Quadratic Regulator (LQR) control theory to design the MIMO controller in JTPC based on the identified system model in Equation (15) LQR is well-suited for this problem because it explicitly accounts for coupling among multiple inputs and outputs while maintaining low runtime overhead. Compared with more computationally expensive control approaches, such as Model Predictive Control (MPC), which requires solving an optimization problem online at every control interval, LQR computes the optimal feedback gain offline and only requires matrix multiplication at runtime. As a result, the controller's computational overhead is negligible, making it practical for embedded real-time systems. The PID control approach could also be used in this application, however, not only does this approach often require precise tuning, but the integral and derivative actions provide no additional utility to the controller in our case. The controller is designed based on the discrete-time state-space system model introduced in Equation (20) in Section III.

We design the LQR controller by minimizing the following quadratic cost function with respect to $\mathbf{u}(k)$ to compute the controller gain $\mathbf{F}$:

$$J = \sum_{k=0}^{\infty} \left(\mathbf{y}(k)^T \mathbf{Q} \mathbf{y}(k) + \mathbf{u}(k)^T \mathbf{R} \mathbf{u}(k) \right), \tag{21}$$

where $\mathbf{Q}$ and $\mathbf{R}$ are weighting matrices that determine the tradeoff between control accuracy and control effort. Specifically, $\mathbf{Q}$ quantifies the cost of state variables deviating from their desired operating points, and a larger $\mathbf{Q}$ leads to a faster response to workload variations. $\mathbf{R}$ specifies the cost of control effort, and the closed-loop system becomes less sensitive to system noise with a larger $\mathbf{R}$. The first term in Equation (21) represents the state regulation cost, which penalizes deviations in task response times and GPU power consumption from their set points. The second term represents the control penalty. Minimizing this term ensures that the controller avoids unnecessary changes to the control inputs, i.e., task rates and GPU frequency, thereby improving system stability.

In JTPC, the internal state vector consists of the response times of the co-scheduled tasks and the GPU power consumption, while the control input vector consists of task-rate adjustments and GPU frequency scaling commands. Although the control objective is specified in terms of RTR, directly modeling RTR as a state variable would introduce additional nonlinear coupling because RTR depends on both response time and task period. Instead, response time is maintained as the internal system state, while RTR is represented through the output equation using the output matrix $\mathbf{C}$ shown in Equation (19), which maps response-time states to the corresponding RTR values.

The optimization problem in Equation (21) is solved *offline* at the beginning of each experiment by solving the discrete-time Riccati recursion in C++, from which the optimal feedback gain matrix $\mathbf{F}$ is obtained. After solving the discrete-time algebraic Riccati equation *offline*, the optimal feedback gain matrix $\mathbf{F}$ is reused. During runtime, the control input vector is computed as $\mathbf{u}(k) = -\mathbf{F}\mathbf{y}(k)$, where the control vector $u(k)$ contains the updated task rates and GPU frequency values applied during the next control interval. The resulting control vector determines the changes in task rates and GPU frequency, where $\mathbf{u}(k) = [\Delta r_1(k)\ \Delta r_2(k)\ \cdots\ \Delta r_N(k)\ \Delta f(k)]^T$. Accordingly, the updated task rates and GPU frequency are applied as $r_i(k+1) = r_i(k) + \Delta r_i(k)$, and $f(k+1) = f(k) + \Delta f(k)$.

C. Stability Analysis

One key benefit of control-theoretic design is that system stability can be analytically guaranteed, even when the operating conditions (e.g., workloads) differ from those used in modeling [9], [22], [23], [41]. We next address the stability analysis of the LQR MIMO controller in the presence of model variations. The overall procedure is summarized as follows:

1) Derive the control input $u(k)$ based on the nominal system model in Equation (15).
2) Inject runtime noise into the system model to emulate modeling inaccuracies and parameter uncertainty, resulting in perturbed system matrices A' and B', as shown in Equation (22a), instead of the nominal matrices A and B in Equation (15).
3) Derive the closed-loop system by substituting the control input $u(k)$ from Equation (22b) into the perturbed system model in Equation (22a). The resulting closed-loop system is given in Equation (22c).
4) Compute the eigenvalues of the closed-loop system matrix in Equation (22c). According to discrete-time control theory, the closed-loop system is stable if the stability condition in Equation (22d) holds, i.e., all eigenvalues lie strictly inside the unit circle in the complex plane.

$$\mathbf{y}(k) = \mathbf{A}'\,\mathbf{y}(k-1) + \mathbf{B}'\,\mathbf{u}(k-1) \tag{22a}$$

$$\mathbf{u}(k) = -\mathbf{K}\,\mathbf{y}(k) \tag{22b}$$

$$\mathbf{y}(k+1) = (\mathbf{A}' - \mathbf{B}'\mathbf{K})\,\mathbf{y}(k) \tag{22c}$$

$$\rho\,(\mathbf{A}' - \mathbf{B}'\mathbf{K}) = \lambda_{\max} < 1 \tag{22d}$$

In our experiments, we perform the above stability analysis across diverse workloads (e.g., varying input sizes) and multiple testbeds. The results show that JTPC remains stable across a broad range of operating conditions. In scenarios where significant model deviations may push the system toward instability, the controller can be re-tuned by updating the system model parameters to better reflect the observed system dynamics, thereby restoring stable operation.

Example. We evaluate the stability of JTPC using the nominal identified model for a configuration with three co-scheduled tasks and one GPU power state. To evaluate robustness against model uncertainty, we perturb the identified input matrix using multiplicative white Gaussian noise, i.e., $\mathbf{B}' = \mathbf{B} \odot (1 + \epsilon\mathcal{N}(0, \sigma^2))$, $\mathbf{A}' = \mathbf{A} \odot (1 + \epsilon\mathcal{N}(0, \sigma^2))$, where $\mathcal{N}(0, \sigma^2)$ denotes zero-mean independent Gaussian noise with variance σ^2, ϵ controls the perturbation magnitude, and $\odot$ denotes element-wise multiplication. The resultant matrices are shown in Equation (23)

$$\mathbf{A}' = 0.95I$$

$$\mathbf{B}' = \begin{bmatrix} 0 & -0.0479 & -0.0520 & 0.0624 \\ -0.0397 & 0 & -0.0556 & 0.0680 \\ -0.0298 & -0.0679 & 0 & 0.0726 \\ 0.0222 & 0.0111 & 0.0285 & -0.0012 \end{bmatrix}. \quad (23)$$

We can then calculate the eigenvalues of the matrix described in step 4. The matrix K is the same as that of the nominal system. The resulting eigenvalues are $0.5669,\ 0.9403,\ 0.8402,\ 0.7482$. Since all eigenvalues lie strictly within the unit circle, the closed-loop system is asymptotically stable. The perturbed system remains stable, demonstrating robustness to model uncertainty.

D. Discussion

Design Consideration. Our framework assumes that the number of tasks, N, is known a priori, which is a practical assumption for many embedded real-time systems where workloads are typically fixed and predefined. If the task count changes, the underlying system dynamics also change, requiring a different controller design based on an updated system model. For platforms that support dynamic task arrivals or departures, one possible extension is to precompute and maintain multiple controller models corresponding to different task configurations, and switch between them at runtime as the workload changes. During design time, equal priority is assigned to timing and power regulation by selecting appropriate weights in the cost function, and the discrete-time Riccati equation is then solved offline to compute the optimal feedback gain matrix. This ensures that the GPU power objective is not assigned a lower priority simply because it corresponds to a single output dimension. This work focuses on time-sliced GPU scheduling, while extending the framework to incorporate power-aware spatial sharing mechanisms remains future work. We assume a context-switch overhead of 750 μs, consistent with the upper bound reported in [8].

V. System Implementation

Hardware Testbed. Our experimental setup uses a Jetson Orin Nano developer kit with CUDA Toolkit 12.0. The controller is implemented in C++. The control period is set to 2 seconds.

LQR MIMO Controller. The controller is implemented as a separate CPU process. Each time its periodic timer fires, the controller sends the average power consumption and the response times of all the tasks. The incoming replies are handled asynchronously so that the controller can avoid being blocked by other applications. After the controller receives the response times and average GPU power, it executes the control algorithm in Section III to compute updated task rates and frequencies.

Rate Modulator. The *rate modulator* runs on each task. Since each task is a process, they receive new task rates from the controller and reset the task timer interval when the task invocation rate changes.

Frequency Modulator. The LQR controller may output a value that does not align with the Jetson's discrete frequency levels (e.g., the Orin Nano supports limited discrete steps). Therefore, the modulator code must locally map the controller's output to a series of supported frequency levels to approximate the desired value. To achieve this, we implement a first-order delta-sigma modulator, a common type of analog-to-digital converter. The detailed algorithm of the first-order delta-sigma modulator can be found in [25], [43]. We adjust the frequency on Jetson devices via the Linux sysfs devfreq interface using the entries exposed under /sys/class/devfreq/17000000.GPU file. A routine periodically checks this file and resets the GPU frequency accordingly.

Power Monitor. To enable runtime power monitoring on the Jetson platform, JTPC employs a dedicated power-monitoring thread that runs in parallel with workload execution and the control loop. GPU power consumption is measured using an external INA260 power sensor [44]. The sensor is sampled at 10 Hz, yielding 20 power samples per control period, which are used to calculate the average GPU power consumption for feedback control.

Baselines. We compare our proposed approach with two baselines. The baselines are (1) FC-GPU [9], which performs rate adaptation to control RTR, and (2) GPU frequency Scaling Only (DFS-only) [45], which performs frequency scaling to control the power consumption.

Metrics. We evaluate performance using the following metrics. The first metric is the RTR, as defined in Equation (1). The second metric is the Deadline Miss Ratio (DMR). We calculate the DMR by recording jobs whose response times exceed their corresponding task periods and marking them as deadline violations. The third metric, Task Utility, is computed as each task's initial period divided by its runtime period, averaged across all tasks. In our experiments, acceptable system performance is achieved when the RTR is within ± 0.02 of the set point, and GPU power stays within the specified power set point.

Workloads. We select workloads from Nvidia Samples and the Rodinia Benchmark for testing. The Nvidia Samples suite includes multiple tasks, of which we use two: `mm`, which performs matrix multiplication, and `stereo`, which uses the GPU to compute pixel-wise disparity between two stereo images for calculating object distance. The Rodinia Benchmark suite provides the remaining tasks: `hist`, which tests the performance of the GPU in handling the memory-intensive task of computing histograms; `quasi`, which implements a quasi-random Monte Carlo pi estimation algorithm on the GPU; and `particle`, which implements the resampling step of a particle filter by selecting particles via cumulative distribution function-based random sampling on the GPU. The specific task configurations used in the experiments are taken from SBEET [6] and SBEET-mg [7].

VI. Evaluation

In this evaluation, we address the following questions. (1) How does `JTPC` compare against the baseline approaches introduced in the motivation study, and can it overcome their limitations through runtime joint control of task rates and GPU frequency (Section VI-A). (2) How effectively does `JTPC` track different `RTR` and power set points while minimizing deadline misses and power violations? (Section VI-B). (3) How robust is `JTPC` under dynamic runtime workload changes, such as the arrival of a new task or changes in control set points? (Section VI-C).

A. Typical Run of `JTPC`

In this subsection, we discuss the limitations of baselines and then discuss how `JTPC` addresses them. To demonstrate the drawback of current *closed-loop* approaches, we choose three periodic tasks with the following configurations: `hist` (117, 160), `stereo` (60, 80), and `mm` (62, 80). The GPU frequency is set to 918 MHz. In this experiment, we demonstrate runtime workload variations for *soft real-time* systems. To simulate a sudden workload increase, during the control period 60, we increase the workload input by altering the problem size of one of the workloads, `mm`.

Figure 4 shows that `FC-GPU` [9] responds to the workload increase by changing the task rates, which reduces the DMR to 0%. However, because it does not regulate GPU power consumption, the framework exceeds the specified set point. `DFS-only` solution controls the GPU power consumption near the specified set point by reducing GPU frequency, as shown in Figure 5. However, because `DFS-only` reduces the operating frequency, execution times increase, causing `RTR` violations and a DMR of 100%. Therefore, neither approach can satisfy both timing and power requirements.

As shown in Figure 6(a) for `JTPC`, the `RTR` remains close to the specified set point before the workload change. When the workload increases at control period 60, the `RTR` temporarily exceeds its set point, leading to deadline misses. However, in subsequent control periods, `JTPC` uses the precomputed LQR feedback gain to jointly adjust task periods and GPU frequency, as shown in Figure 6(c), enabling the system to quickly recover. As a result, the deadline miss ratio is 0% in the subsequent control period shown in Figure 6(b) and the `RTR` converges back to the desired set point, and GPU power consumption is controlled close to the specified power set point, as shown in Figure 6(d).

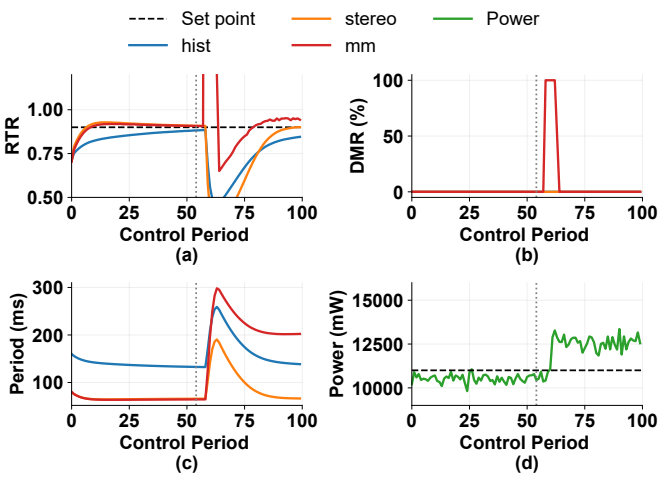


Fig. 4: The Performance of `FC-GPU` on the Jetson Orin Nano. The closed-loop baseline under a runtime workload increase at control period 60: (a) `RTR`, (b) DMR, (c) task period adaptation, and (d) GPU power consumption.

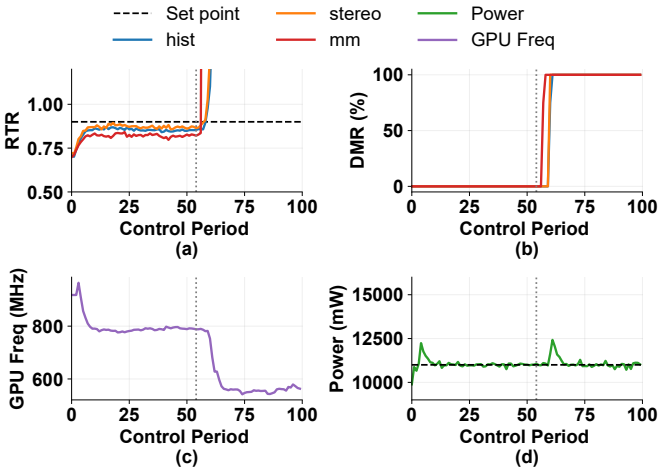


Fig. 5: Performance of the DFS closed-loop baseline under a runtime workload increase at control period 60: (a) `RTR`, (b) DMR, (c) GPU frequency adaptation, and (d) GPU power consumption.

B. Control Accuracy under Different Set Points

In this section, we evaluate the performance of `JTPC` across a range of `RTR` and power set points. As observed in previous experiments, we compute the average `RTR` and power consumption, along with their standard deviations, after the system reaches a steady state to mitigate the influence of transient states at the beginning of each run. We first test different `RTR` set points from 0.70 to 0.90 with an interval of 0.05, while keeping the power set point fixed at a typical

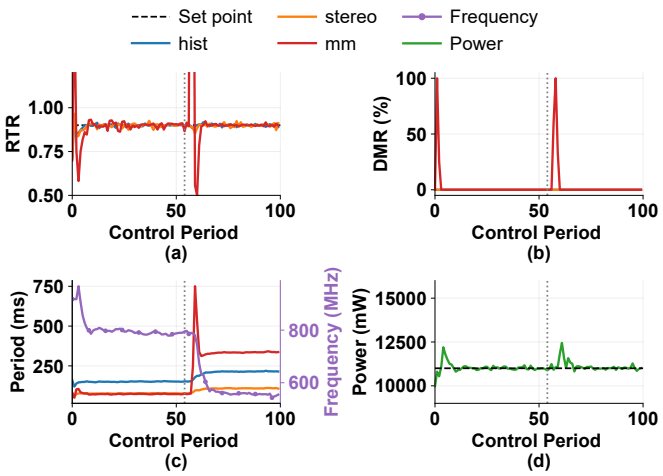
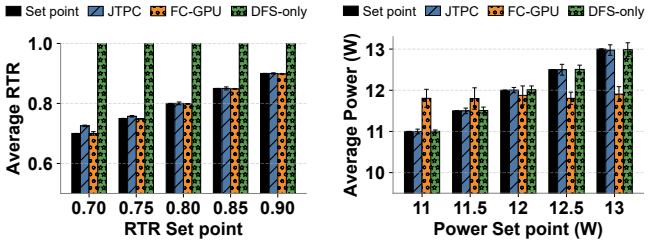


Fig. 6: Control response of JTPC under a runtime workload increase at control period 60: (a) RTR, (b) DMR, (c) task period and GPU frequency adaptation, (d) GPU power consumption. JTPC jointly controls both RTR and GPU power. As a result, it keeps the RTR close to the specified set point while the DMR returns to 0% after the transient, ensuring GPU power stays within the specified power set point .



(a) RTR (b) Power consumption

Fig. 7: Set point tracking under single-parameter sweeps: (a) RTR tracking with fixed 11 W power and (b) power tracking with fixed 0.90 RTR.

value of 11 W. We then test different power set points from 11 W to 13 W with an interval of 0.5, while keeping the RTR set point fixed at 0.90. For both experiments, we use the same task configurations as in Section VI-A.

Figure 7a shows the RTR for each solution for different set points. Both JTPC and FC-GPU consistently track the target RTR with minimal deviation. In addition, both solutions maintain a DMR of 0%. DFS-only approach exhibits the largest deviation in RTR, as shown in Figure 7a, because it regulates only GPU power consumption and does not directly control task response times, which leads to deadline misses of up to 13.51% at some set points. However, FC-GPU does not explicitly regulate GPU power consumption, resulting in higher power consumption for lower set points and lower task utility for higher set points, as shown in Figures 7b and 8. DFS-only has utility close to the initial task rates because it does not perform rate adaptation. FC-GPU has constant task utility because it regulates only RTR; therefore, changes to the power set point do not affect task utility.

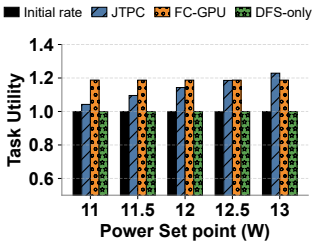


Fig. 8: Average task utility for a fixed RTR set point (0.90) and different power set points.

In contrast, JTPC increases task utility as the power set point relaxes because it includes power error in the feedback loop. For JTPC with a higher power set point, the LQR controller exploits the increased power budget to optimally balance higher GPU frequency and shorter task periods, maximizing utility as shown in Figure 8, while regulating power near the desired set point. JTPC controls the GPU power consumption close to the set point as shown in Figure 7b. Although DFS-only controls the power consumption close to the specific set point, it fails to satisfy timing constraints. To further interpret the power-sweep result, Figure 8 reports the task utility metric under the same fixed-RTR setting used in the power-tracking experiment. Here, we fix the RTR set point at 0.90 and sweep the power set point from 11 W to 13 W in 0.5 W increments. We compute the task utility metric as each task's initial period divided by its runtime period, averaged across the three tasks. Since the initial periods remain fixed across set points, this metric captures the normalized task-rate improvement under different power budgets.

C. Arrival of a new task

Real-time embedded GPU systems may experience dynamic changes in workload due to evolving application demands, such as the activation of additional tasks at runtime. For example, in an autonomous driving system, a new perception task such as obstacle detection may need to be launched in response to unexpected environmental events. The arrival of a new task increases GPU contention and power consumption, potentially leading to higher task response times, deadline misses, and power-budget violations. As discussed in Section IV-D, JTPC handles such scenarios by switching between precomputed task configurations. To evaluate this capability, we begin with three periodic GPU tasks hist (117, 160), stereo (60, 80), and mm (62, 80) and introduce a fourth task particle (79.3, 100) at control period 60.

Figure 9(a) shows that the RTR of the active tasks remains close to the desired set point before the new task arrives. At control period 60, when the additional task is introduced, the increased workload causes a temporary rise in the outputs, as shown in Figure 9(b). In response, JTPC dynamically adjusts both inputs, as shown in Figure 9(c), to compensate for the increased system load. As a result, the RTR quickly converges back to the desired set point, while the deadline miss ratio returns to near zero. Simultaneously, GPU power consumption remains close to the specified 11 W power set point, as shown in Figure 9(d). These results demonstrate that JTPC can effectively handle dynamic task arrivals by jointly regulating timing and power objectives at runtime.

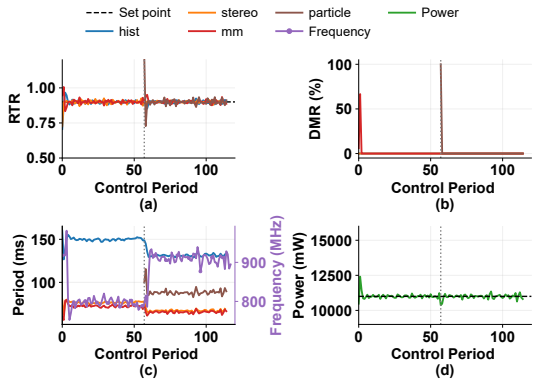


Fig. 9: (a) RTR, (b) DMR, (c) task period and GPU frequency adaptation, and (d) GPU power consumption. At control period 60, when the new task arrives, the RTR and DMR temporarily increases. JTPC then jointly adjusts task rates and frequency to control the RTR and GPU power consumption to the set point.

VII. Related Work

The related work in this paper is categorized into three categories: 1) *Feedback Performance Control in Computing Systems*, 2) *Real-Time GPU Scheduling*, and 3) *Power Control*.

Feedback performance control. As a theoretical tool for designing and analyzing adaptive systems, feedback control theory has been extensively applied to computer performance management [46]. For example, several control-theoretic techniques have been developed to control the CPU temperature or power consumption [42], [45], [47]. Some projects have applied control theory to real-time scheduling and applications. For example, Steere et al. have developed a feedback-based scheduler [48] that guarantees desired progress rates for real-time applications. Abeni et al. presented a control analysis of a reservation-based feedback scheduler [49]. The most closely related work is Feedback Control Scheduling (FCS) [28], [30], which dynamically adjusts the invocation rates of periodic real-time tasks to control either the CPU utilization or the task deadline miss ratio. For distributed real-time systems, end-to-end CPU utilization control [32], [50] and decentralized utilization control algorithms [31] have also been proposed. In addition, FCS has been applied to various real-time applications, including autonomous driving [51] [3], information dissemination [52], [53], and smart grids [54].

Real-Time GPU Scheduling. The increasing popularity of general-purpose GPU computing creates a need for more advanced scheduling algorithms that accommodate diverse applications while ensuring full utilization of available hardware. The TimeGraph framework presented by Kato et al. [55], GPU-TDMh proposed by Golyanik et al. [16], and RT-GPU proposed by Zhou et al. [56] implement scheduling algorithms designed for hard real-time systems that use GPUs. There is also a considerable amount of work designed to enable efficient GPU task preemption, such as PipeSwitch [57], EffiSha [58], REEF [35], GPREEMPT [59], Padel [15], and GCAPS [8]. Recently, a closed-loop solution called FC-GPU was proposed, which incorporates runtime feedback for adaptive scheduling; however, FC-GPU does not provide runtime power control.

Power Control. Bhatti et al. [60] present a general power management framework for embedded systems executing periodic real-time tasks. Ring-DVFS [61] employs reinforcement learning to reduce power consumption while maintaining task reliability. Liu et al. [62] develop an offline DVFS planning approach to reduce the thermal footprint of embedded devices. Wang et al. proposed SBEET [6] and SBEET-mg [7], which combines offline task-to-GPU allocation, runtime job migration, and SM-level spatial multitasking to reduce both energy consumption and deadline misses.

Table I provides a comprehensive comparison of existing solutions and shows that JTPC is the first framework that performs feedback control real-time GPU scheduling with power control.

Solutions	Real-time	Adaptability	Power Control
Padel [15]	✓	×	×
TimeGraph [55]	✓	×	×
GPU-TDMh [16]	✓	×	×
SBEET [6]	✓	×	×
SBEET-mg [7]	✓	×	×
FC-GPU [9]	✓	✓	×
JTPC	✓	✓	✓

TABLE I: Comparison of JTPC with existing Solutions.

VIII. Conclusion

Embedded GPUs are increasingly used in real-time applications, but they must operate under strict power and thermal constraints while still satisfying task timing requirements. Existing solutions typically regulate either task timeliness or GPU power consumption independently, which can lead to deadline misses or power-budget violations because task-rate adaptation and GPU frequency scaling are inherently coupled. In this paper, we presented JTPC, a joint timeliness and power-control framework for soft real-time embedded GPU systems. JTPC models the interaction between task response times, task rates, GPU frequency, and GPU power consumption using a MIMO system model, and designs an LQR-based controller to coordinate task-rate adaptation and dynamic frequency scaling. Experimental results on an Nvidia Jetson platform show that JTPC tracks both RTR and power set points with minimal oscillations.

References

[1] Z. Li, N. Didar, and M. Brocanelli, "Offloading-aware control of ai task allocation and virtual object quality in mar apps," *IEEE Transactions on Mobile Computing*, 2025.

[2] M. Fernández-Sanjurjo, M. Mucientes, and V. M. Brea, "Real-time multiple object visual tracking for embedded gpu systems," *IEEE Internet of Things Journal*, vol. 8, no. 11, pp. 9177–9188, 2021.

[3] Y. Bai, Z. Wang, X. Wang, and J. Wang, "Autoe2e: End-to-end real-time middleware for autonomous driving control," in *2020 IEEE 40th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2020, pp. 1101–1111.

[4] H. Kim, P. Patel, S. Wang, and R. R. Rajkumar, "A server-based approach for predictable gpu access control," in *2017 IEEE 23rd International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA)*. IEEE, 2017, pp. 1–10.

[5] S. K. Saha, Y. Xiang, and H. Kim, "Stgm: Spatio-temporal gpu management for real-time tasks," in *2019 IEEE 25th International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA)*. IEEE, 2019, pp. 1–6.

[6] Y. Wang, M. Karimi, Y. Xiang, and H. Kim, "Balancing energy efficiency and real-time performance in gpu scheduling," in *2021 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2021, pp. 110–122.

[7] Y. Wang, M. Karimi, and H. Kim, "Towards energy-efficient real-time scheduling of heterogeneous multi-gpu systems," in *2022 IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2022, pp. 409–421.

[8] Y. Wang, C. Liu, D. Wong, and H. Kim, "Gcaps: Gpu context-aware preemptive priority-based scheduling for real-time tasks," in *36th Euromicro Conference on Real-Time Systems*, 2024.

[9] S. Subramaniyan and X. Wang, "Fc-gpu: Feedback control gpu scheduling for real-time embedded systems," *ACM Transactions on Embedded Computing Systems*, vol. 24, no. 5s, pp. 1–25, 2025.

[10] S. Subramaniyan, R. Joshi, X. Wang, and M. Brocanelli, "Seeb-gpu: Early-exit aware scheduling and batching for edge gpu inference," in *Proceedings of the Tenth ACM/IEEE Symposium on Edge Computing*, 2025, pp. 1–16.

[11] N. Otterness, M. Yang, T. Amert, J. Anderson, and F. D. Smith, "Inferring the scheduling policies of an embedded cuda gpu," in *Workshop on Operating Systems Platforms for Embedded Real Time Systems Applications (OSPERT)*, 2017.

[12] N. Otterness and J. H. Anderson, "Exploring amd gpu scheduling details by experimenting with "worst practices"," in *29th International Conference on Real-Time Networks and Systems*, 2021.

[13] J. Bakita and J. H. Anderson, "Demystifying nvidia gpu internals to enable reliable gpu management," 2024.

[14] T. Amert, N. Otterness, M. Yang, J. H. Anderson, and F. D. Smith, "Gpu scheduling on the nvidia tx2: Hidden details revealed," in *IEEE Real-Time Systems Symposium (RTSS)*. IEEE, 2017.

[15] A. Gheibi-Fetrat, S. Safari, A. Ahmadi-Tonekaboni, S. Hessabi, and H. Sarbazi-Azad, "Padel: Priority-based real-time scheduling for gpus," *IEEE Embedded Systems Letters*, 2025.

[16] V. Golyanik *et al.*, "Towards scheduling hard real-time image processing tasks on a single gpu," in *ICIP*. IEEE, 2017.

[17] NVIDIA Corporation, *Jetson AGX Orin Series Thermal Design Guide*, NVIDIA Corporation, 2022, version 1.1.

[18] S. Kim, K. Bin, S. Ha, K. Lee, and S. Chong, "ztt: Learning-based dvfs with zero thermal throttling for mobile devices," *GetMobile: Mobile Computing and Communications*, vol. 25, no. 4, pp. 30–34, 2022.

[19] S. Hosseinimotlagh, A. Ghahremannezhad, and H. Kim, "On dynamic thermal conditions in mixed-criticality systems," in *2020 IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*. IEEE, 2020, pp. 336–349.

[20] Y. Lee, K. G. Shin, and H. S. Chwa, "Thermal-aware scheduling for integrated cpus–gpu platforms," *ACM Transactions on Embedded Computing Systems (TECS)*, vol. 18, no. 5s, pp. 1–25, 2019.

[21] G. Chen and X. Wang, "Performance optimization of machine learning inference under latency and server power constraints," in *2022 IEEE 42nd International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2022, pp. 325–335.

[22] Y. Ma, S. Subramaniyan, and X. Wang, "Power capping of gpu servers for machine learning inference optimization," in *Proceedings of the 54th International Conference on Parallel Processing*, 2025, pp. 449–459.

[23] G. Chen and X. Wang, "Performance optimization of machine learning inference under latency and server power constraints," in *2022 IEEE 42nd International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2022, pp. 325–335.

[24] S. Subramaniyan and X. Wang, "Cats: Correlation-aware task scheduling for gpu power optimization in ai data centers," in *Proceedings of the International Conference on Supercomputing (ICS '26)*. Belfast, United Kingdom: Association for Computing Machinery, 2026.

[25] S. Subramaniyan, B. Morse, and X. Wang, "PowerGPU: Power-aware feedback control gpu scheduling in real-time embedded systems," in *Proceedings of the 44th IEEE International Conference on Computer Design (ICCD)*. Hong Kong, China: IEEE, 2026.

[26] L. Sha, R. Rajkumar, and S. S. Sathaye, "Generalized rate-monotonic scheduling theory: A framework for developing real-time systems," *Proceedings of the IEEE*, vol. 82, no. 1, pp. 68–82, 1994.

[27] N. Otterness, M. Yang, S. Rust, E. Park, J. H. Anderson, F. D. Smith, A. Berg, and S. Wang, "An evaluation of the nvidia tx1 for supporting real-time computer-vision workloads," in *Real-Time and Embedded Technology and Applications Symposium*. IEEE, 2017.

[28] C. Lu, J. A. Stankovic, G. Tao, and S. H. Son, "Feedback control real-time scheduling: Framework, modeling, and algorithms," *Journal of Real-Time Systems, Special Issue on Control-Theoretical Approaches to Real-Time Computing*, vol. 23, no. 1/2, pp. 85–126, Jul. 2002.

[29] J. Yao, X. Liu, M. Yuan, and Z. Gu, "Online adaptive utilization control for real-time embedded multiprocessor systems," in *Proceedings of the 6th IEEE/ACM/IFIP international conference on Hardware/Software codesign and system synthesis*, 2008.

[30] C. Lu, X. Wang, and C. Gill, "Feedback control real-time scheduling in ORB middleware," in *Proceedings of 9th IEEE Real-Time and Embedded Technology and Applications Symposium (RTAS)*, 2003.

[31] X. Wang, D. Jia, C. Lu, and X. Koutsoukos, "DEUCON: Decentralized end-to-end utilization control for distributed real-time systems," *IEEE Transactions on Parallel and Distributed Systems*, vol. 18, no. 7, pp. 996–1009, 2007.

[32] C. Lu, X. Wang, and X. Koutsoukos, "End-to-end utilization control in distributed real-time systems," in *24th International Conference on Distributed Computing Systems, 2004. Proceedings.*, 2004.

[33] J. Guo and C. Peng, "Adaptive tracking: Dynamic frame rate for drone-based target following," in *2024 IEEE 15th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON)*. IEEE, 2024, pp. 0278–0285.

[34] G. C. Buttazzo, G. Lipari, and L. Abeni, "Elastic task model for adaptive rate control," in *Proceedings 19th IEEE Real-Time Systems Symposium (Cat. No. 98CB36279)*. IEEE, 1998, pp. 286–295.

[35] M. Han, H. Zhang, R. Chen, and H. Chen, "Microsecond-scale preemption for concurrent GPU-accelerated DNN inferences," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 539–558.

[36] R. A. Bridges, N. Imam, and T. M. Mintz, "Understanding gpu power: A survey of profiling, modeling, and simulation methods," *ACM Computing Surveys (CSUR)*, vol. 49, no. 3, pp. 1–27, 2016.

[37] S. Hong and H. Kim, "An integrated gpu power and performance model," in *Proceedings of the 37th annual international symposium on Computer architecture*, 2010, pp. 280–289.

[38] J. Guerreiro, A. Ilic, N. Roma, and P. Tomás, "Dvfs-aware application classification to improve gpgpus energy efficiency," *Parallel Computing*, vol. 83, pp. 93–117, 2019.

[39] R. Ge, R. Vogt, J. Majumder, A. Alam, M. Burtscher, and Z. Zong, "Effects of dynamic voltage and frequency scaling on a k20 gpu," in *2013 42nd international conference on parallel processing*. IEEE, 2013, pp. 826–833.

[40] G. Chen, S. Subramaniyan, and X. Wang, "Latency-guaranteed co-location of inference and training for reducing data center expenses," in *2024 IEEE 44th International Conference on Distributed Computing Systems (ICDCS)*. IEEE, 2024, pp. 473–484.

[41] G. Chen and X. Wang, "Optimml: Joint control of inference latency and server power consumption for ml performance optimization," *ACM Transactions on Autonomous and Adaptive Systems*, vol. 21, no. 2, pp. 1–27, 2026.

[42] K. Ma, X. Li, M. Chen, and X. Wang, "Scalable power control for many-core architectures running multi-threaded applications," in *Proceedings of the 38th annual international symposium on Computer architecture*, 2011, pp. 449–460.

[43] X. Wang and M. Chen, "Cluster-level feedback power control for performance optimization," in *2008 IEEE 14th International Symposium on High Performance Computer Architecture*. IEEE, 2008, pp. 101–110.

[44] M. Karimi, Y. Wang, and H. Kim, "An open-source power monitoring framework for real-time energy-aware gpu scheduling research," in *Open Demo Session of IEEE Real-Time Systems Symposium (RTSS@ Work)*, 2022.

[45] Y. Wang, K. Ma, and X. Wang, "Temperature-constrained power control for chip multiprocessors with online model estimation," *SIGARCH Comput. Archit. News*, vol. 37, no. 3, p. 314–324, Jun. 2009.

[46] J. L. Hellerstein, Y. Diao, S. Parekh, and D. M. Tilbury, *Feedback Control of Computing Systems*. John Wiley & Sons, 2004.

[47] K. Skadron, T. Abdelzaher, and M. R. Stan, "Control-theoretic techniques and thermal-RC modeling for accurate and localized dynamic thermal management," in *Proceedings of the 8th International Symposium on High-Performance Computer Architecture (HPCA)*, 2002.

[48] D. C. Steere, A. Goel, J. Gruenberg, D. McNamee, C. Pu, and J. Walpole, "A feedback-driven proportion allocator for real-rate scheduling," in *Proceedings of the 3rd Operating Systems Design and Implementation (OSDI)*, 1999.

[49] L. Abeni, L. Palopoli, G. Lipari, and J. Walpole, "Analysis of a reservation-based feedback scheduler," in *Proceedings of the 23rd IEEE Real-Time Systems Symposium (RTSS)*, Dec. 2002.

[50] C. Lu, X. Wang, and X. Koutsoukos, "Feedback utilization control in distributed real-time systems with end-to-end tasks," *IEEE transactions on parallel and distributed systems*, vol. 16, no. 6, pp. 550–561, 2005.

[51] Y. Bai, L. Li, Z. Wang, X. Wang, and J. Wang, "Performance optimization of autonomous driving control under end-to-end deadlines," *Real-Time Systems*, vol. 58, no. 4, pp. 509–547, 2022.

[52] M. Chen, X. Wang, and B. Taylor, "Integrated control of matching delay and cpu utilization in information dissemination systems," in *Proceedings of the 17th IEEE International Workshop on Quality of Service (IWQoS)*, 2009.

[53] M. Chen, X. Wang, R. Gunasekaran, H. Qi, and M. Shankar, "Control-based real-time metadata matching for information dissemination," in *Proceedings of the 14th IEEE International Conference on Embedded and Real-Time Computing Systems and Applications (RTCSA)*, 2008.

[54] M. Chen, X. Wang, and C. Nolan, "Hierarchical cpu utilization control for real-time guarantees in power grid computing," *Journal of Real-Time Systems*, vol. 48, no. 2, pp. 198–221, Jan. 2012.

[55] S. Kato *et al.*, "Timegraph: Gpu scheduling for real-time multi-tasking environments," in *USENIX ATC*, 2011.

[56] A. Zou, J. Li, C. D. Gill, and X. Zhang, "Rtgpu: Real-time gpu scheduling of hard deadline parallel tasks with fine-grain utilization," *IEEE Transactions on Parallel and Distributed Systems*, vol. 34, no. 5, pp. 1450–1465, 2023.

[57] Z. Bai, Z. Zhang, Y. Zhu, and X. Jin, "PipeSwitch: Fast pipelined context switching for deep learning applications," in *Proceedings of the 14th USENIX Conference on Operating Systems Design and Implementation (OSDI)*, 2020.

[58] G. Chen, Y. Zhao, X. Shen, and H. Zhou, "Effisha: A software framework for enabling effficient preemptive scheduling of gpu," in *Proceedings of the 22nd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, 2017, pp. 3–16.

[59] R. Fan, T. Ren, M. Xie, S. Gao, J. Shu, and Y. Lu, "{GPREEMPT}:{GPU} preemptive scheduling made general and efficient," in *2025 USENIX Annual Technical Conference (USENIX ATC 25)*, 2025, pp. 263–272.

[60] K. Bhatti, C. Belleudy, and M. Auguin, "Power management in real time embedded systems through online and adaptive interplay of dpm and dvfs policies," in *2010 IEEE/IFIP International Conference on Embedded and Ubiquitous Computing*. IEEE, 2010, pp. 184–191.

[61] A. Yeganeh-Khaksar, M. Ansari, S. Safari, S. Yari-Karin, and A. Ejlali, "Ring-dvfs: Reliability-aware reinforcement learning-based dvfs for real-time embedded systems," *IEEE Embedded Systems Letters*, vol. 13, no. 3, pp. 146–149, 2020.

[62] Y. Liu, H. Yang, R. P. Dick, H. Wang, and L. Shang, "Thermal vs energy optimization for dvfs-enabled processors in embedded systems," in *8th International Symposium on Quality Electronic Design (ISQED'07)*. IEEE, 2007, pp. 204–209.